Java OOP principles

Contents

What object-oriented programming is, why it exists, the four pillars, and the everyday building blocks like classes, constructors, and inheritance. Definitions stay tight; the answer does the teaching.

1. What is Object-Oriented Programming (OOP)?

OOP is a programming approach where software is built using objects. Each object holds its own data (fields) and the methods that work on that data. Grouping the two together is what keeps the code reusable, easier to maintain, and well organised.

2. Why was OOP introduced, and what problems does it solve?

Why it was introduced: before OOP, most applications were written procedurally, as functions operating on shared data. That worked fine until the codebase grew. Then the shared-data model started to hurt: functions and data were tightly coupled, logic got duplicated, changes in one place broke another, and the whole thing became hard to maintain and scale. OOP answered this by grouping data and the operations on it into objects, so each object owns its own state.

What it solves:

Problem	How OOP solves it
Code duplication	**Inheritance** lets one class reuse another's code
Tight coupling	**Abstraction** and **interfaces** cut dependencies between parts
Data corruption	**Encapsulation** hides internal state and controls access to it
Hard to extend	**Polymorphism** lets us add new implementations with minimal change
Poor maintainability	Code is organised into classes with clear responsibilities
Poor scalability	A modular design keeps large systems manageable

3. What are the four pillars of OOP?

The four pillars are **encapsulation, abstraction, inheritance, and polymorphism**. Together they are what make an OOP system modular, reusable, and easy to extend.

Encapsulation: bundle the data and its methods together, and restrict direct access to the internal state. In Java we do this with private fields and public methods (getters and setters).

```
class BankAccount {
    private double balance; // hidden data

    public void deposit(double amount) {
        balance += amount;
    }

    public double getBalance() {
        return balance;
    }
}
```

Abstraction: hide the implementation detail and expose only what the caller needs. Interfaces and abstract classes are how we get it.

```
interface Payment {
    void pay(double amount);
}

class CreditCardPayment implements Payment {
    @Override
    public void pay(double amount) {
        System.out.println("Paid using credit card");
    }
}
```

The caller only ever writes `payment.pay(1000)`. It does not need to know how the payment is actually processed.

Inheritance: let one class take on the properties and behaviour of another. This gives us code reuse and an IS-A relationship.

```
class Animal {
    void eat() {
        System.out.println("Eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Barking");
    }
}
```

Polymorphism: let the same method call do different things depending on the object behind it. We get it two ways: overriding (runtime) and overloading (compile-time).

Runtime polymorphism, through overriding:

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Bark");
    }
}

Animal animal = new Dog();
animal.sound(); // Bark
```

Compile-time polymorphism, through overloading:

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }
}
```

4. Why is Java considered an object-oriented language?

Java is considered object-oriented because it **organises programs around classes and objects** and supports all four pillars: encapsulation, abstraction, inheritance, and polymorphism.

What makes it object-oriented:

- Everything is organised into classes and objects.
- Objects encapsulate state (fields) and behaviour (methods).
- Encapsulation comes from access modifiers.
- Abstraction comes from interfaces and abstract classes.
- Inheritance comes from `extends` and `implements`.
- Polymorphism comes from method overriding and overloading.

Worth being precise, though: Java is **not a purely** object-oriented language, because it also has primitive types like `int`, `double`, `char`, `boolean`, `byte`, `short`, `long`, and `float`. Those are not objects. In a purely object-oriented language, everything would be an object.

5. Why are primitive types still present in Java?

For **performance and memory efficiency**. Primitives let us work with simple values like numbers and booleans without paying the cost of creating a full object for each one.

6. What are the advantages and disadvantages of OOP?

Advantages:

Advantage	Explanation
Code reusability	Inheritance and composition let us reuse existing code and cut duplication
Encapsulation	Protects internal data by exposing only what is needed through methods
Abstraction	Hides implementation detail and exposes only the required functionality
Polymorphism	One interface can have many implementations, so the code flexes and extends easily
Maintainability	Related data and behaviour sit together in a class, so changes are easier
Modularity	The application splits into independent classes that are easier to follow
Testability	Classes with clear responsibilities are easier to unit test and mock

Advantage	Explanation
Scalability	A well-designed OOP app is easier to extend as requirements grow

Disadvantages:

Disadvantage	Explanation
Higher complexity	Small problems get over-engineered into too many classes and abstractions
Performance overhead	Object creation, method dispatch, and garbage collection cost more than procedural code
Higher memory usage	Objects carry metadata and references, so they use more memory than raw values
Inheritance misuse	Deep hierarchies create tight coupling and get hard to maintain
Learning curve	Abstraction, polymorphism, and design patterns take time to grasp
Over-abstraction	Too many interfaces and layers make simple code hard to follow
Debugging complexity	Tracing a call across many objects and layers can be harder

7. Procedural programming v/s object-oriented programming.

Procedural organises a program as a sequence of functions that operate on data. The focus is on *how* the task is performed.

```
// procedural style
class SalaryCalculator {
    public static double calculateSalary(String role) {
        if ("Developer".equals(role)) {
            return 100000;
        } else if ("Manager".equals(role)) {
            return 150000;
        }
        return 0;
    }
}
```

The trouble shows up as the app grows: long `if-else` or `switch` chains, a change needed every time a new role is added, and logic scattered across the code.

Object-oriented organises the program into objects, where each object holds both its data and its behaviour. The focus shifts to *what* the object does.

```java
abstract class Employee {
    abstract double calculateSalary();
}

class Developer extends Employee {
    @Override
    double calculateSalary() {
        return 100000;
    }
}

class Manager extends Employee {
    @Override
    double calculateSalary() {
        return 150000;
    }
}
```

Now adding a new role means writing a new class, not editing existing code. That is the real difference: procedural spreads the decision across one growing function, OOP pushes each case into its own object.

8. What is a class and an object?

Class: a blueprint or template for creating objects. It defines the properties (fields) and behaviours (methods) that objects of that type will have.

```java
class Car {
    String brand;           // property

    void start() {     // behaviour
        System.out.println("Car started");
    }
}
```

Object: a runtime instance of a class. It has its own state (the field values) and can call the methods its class defines.

```java
public class Main {
    public static void main(String[] args) {
        Car car = new Car();   // object creation
        car.brand = "BMW";     // state
        car.start();           // behaviour
    }
}
```

So `Car` is the class (the blueprint) and `car` is the object (one instance of it).

9. Getters v/s setters.

A **getter** reads the value of a private field; a **setter** updates it. Together they support encapsulation by keeping the field private and routing all access through methods we control.

```
class Employee {
    private String name;
    private int age;

    // getter
    public String getName() {
        return name;
    }

    // setter
    public void setName(String name) {
        this.name = name;
    }

    // getter
    public int getAge() {
        return age;
    }

    // setter
    public void setAge(int age) {
        this.age = age;
    }
}
```

Using it:

```
Employee emp = new Employee();

emp.setName("Rohit");
emp.setAge(27);

System.out.println(emp.getName());
System.out.println(emp.getAge());
```

10. How does Java achieve abstraction?

By hiding the implementation detail and exposing only the essential functionality. In practice we get there two ways: interfaces and abstract classes.

Using interfaces (preferred): an interface says *what* a class should do, not *how* it does it.

```java
interface Payment {
    void pay(double amount);
}

class CreditCardPayment implements Payment {
    @Override
    public void pay(double amount) {
        System.out.println("Paid using credit card");
    }
}

class UPIPayment implements Payment {
    @Override
    public void pay(double amount) {
        System.out.println("Paid using UPI");
    }
}
```

```java
Payment payment = new UPIPayment();
payment.pay(1000);
```

The caller knows what to call (`pay()`) but not how the payment is processed.

Using abstract classes: an abstract class can hold both abstract methods (no body) and concrete ones (with a body).

```java
abstract class Animal {
    abstract void sound();

    void sleep() {
        System.out.println("Sleeping");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Bark");
    }
}
```

```java
Animal animal = new Dog();
animal.sound();
animal.sleep();
```

11. Common mistakes when applying abstraction.

Mistake	Why it is a problem	Better approach
Interface for every class	Adds complexity when there is only one implementation	Add an interface only when we expect several implementations or need a clear contract
Leaking implementation detail	Clients get coupled to the internals, which defeats abstraction	Expose the operations, not how they are implemented
God interfaces	A huge interface breaks the Interface Segregation Principle	Split it into smaller, focused interfaces
Over-abstraction	Too many layers and interfaces make the code hard to navigate	Keep abstractions simple and meaningful
Wrong abstraction level	Mixing business logic with infrastructure makes code harder to maintain	Separate business logic from persistence, networking, and so on
Coupling to concrete classes	Testing and swapping implementations gets difficult	Depend on interfaces or abstractions where it makes sense
Ignoring the domain	Generic abstractions like `BaseService` or `BaseManager` do not reflect the business	Model abstractions around business capabilities
Inheritance instead of abstraction	Deep inheritance hierarchies create tight coupling	Prefer interfaces and composition unless there is a true IS-A relationship

12. Types of inheritance supported by Java.

Type	Supported	Description
Single	Yes	One child class inherits from one parent
Multilevel	Yes	A class inherits from another child class, forming a chain
Hierarchical	Yes	Several child classes inherit from the same parent
Multiple (classes)	No	A class cannot inherit from more than one class
Hybrid (classes)	No	Any combination that involves multiple class inheritance is not allowed

Single: one child inherits from one parent.

```
class Animal {
    void eat() {
        System.out.println("Eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Barking");
    }
}
```

Multilevel: a child becomes the parent of another class, so it forms a chain.

```
class Animal {
    void eat() {
        System.out.println("Eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Barking");
    }
}

class Puppy extends Dog {
    void play() {
        System.out.println("Playing");
    }
}
```

Hierarchical: several child classes inherit from one parent.

```
class Animal {
    void eat() {
        System.out.println("Eating");
    }
}

class Dog extends Animal {
}

class Cat extends Animal {
}
```

Multiple (not supported for classes): a class cannot extend two classes.

```
class A {
}

class B {
}

// compilation error
class C extends A, B {
}
```

Java blocks this to avoid the diamond problem: if both parents defined the same method, the compiler would not know which one `C` should inherit.

How Java still gives us multiple inheritance: through interfaces. A class can implement as many as it needs.

```
interface Flyable {
    void fly();
}

interface Swimmable {
    void swim();
}

class Duck implements Flyable, Swimmable {
    public void fly() {
    }

    public void swim() {
    }
}
```

13. What is a constructor, and what are its types?

A constructor is a **special method that initialises an object when it is created**. It carries the same name as the class, has no return type, and runs automatically when we create the object with `new`.

```
class Car {
    Car() {
        System.out.println("Car object created");
    }
}

public class Main {
    public static void main(String[] args) {
        Car car = new Car(); // constructor is called automatically
    }
}
```

Types: Java mainly has three.

Type	Description
Default constructor	Added automatically by the compiler when we write no constructor at all
No-argument constructor	Written by us, but takes no parameters
Parameterised constructor	Takes parameters, so the object starts with specific values

Many interviewers consider there to be only two types:

- Default constructor
- Parameterised constructor

14. this() v/s super().

Both are constructor calls. `this()` calls another constructor in the same class; `super()` calls a constructor in the parent class.

Feature	`this()`	`super()`
Calls	Another constructor in the same class	A constructor of the parent class
Purpose	Constructor chaining within the class	Initialise the parent part of the object
Target	Current class	Parent class
Position	Must be the first statement	Must be the first statement

Since each must be the first statement, we **cannot use both in the same constructor**. We pick one.

15. What coding standards would you establish to keep object-oriented code scalable, testable, and maintainable over many years?

The way I think about it, most of these are about keeping change cheap: the code we write today should not become the thing nobody wants to touch in three years. So the standards I would hold the line on:

1. **Follow SOLID.** Single responsibility, open/closed, Liskov substitution, interface segregation, and dependency inversion. These are the backbone.
2. **Prefer composition over inheritance.** Use inheritance only for a true IS-A relationship; reach for composition when we want flexibility and loose coupling.
3. **Use constructor injection.** Avoid field injection, and make the required dependencies `final`.
4. **Keep classes focused.** One class, one primary responsibility. No God classes.

5. **Program to interfaces.** Depend on abstractions, not concrete implementations.
6. **Encapsulate data.** Keep fields private and expose only the behaviour that is actually needed.
7. **Favour immutability.** Use `final` fields where we can, and minimise mutable shared state.
8. **Write clean, readable code.** Meaningful names, small methods, no duplication (DRY), and no clever-for-the-sake-of-it code.
9. **Design for testability.** Keep business logic independent of frameworks, mock external dependencies, and write both unit and integration tests.
10. **Handle exceptions consistently.** Throw meaningful exceptions, never swallow them, and use global exception handling in web apps.
11. **Avoid premature abstraction.** Do not add an interface or a layer until it earns its place.
12. **Document intent.** Aim for self-explanatory code first, and add a comment only when the reasoning is not obvious from the code.

If I had to compress it to one line: keep responsibilities small, depend on abstractions, and make the code easy to test. The rest tends to follow from those three.